EAD
EDUCAÇÃO A DISTÂNCIA

UNIP
UNIVERSIDADE PAULISTA

Roteiros

Banco de Dados

Instituto de Ciências Exatas e Tecnológicas	**Disciplina:** Banco de Dados **Título da Aula:** Introdução e Fundamentação	**ROTEIRO 1**

Roteiro da aula prática – Introdução e Fundamentação.

1. Objetivos da aula

- Compreender a evolução e a importância dos dados no contexto organizacional.
- Explorar a Atividade Teórico-Prática, dirigida sobre o histórico dos bancos de dados, desde os conceitos fundamentais até sistemas de gerenciamento de banco de dados relacionais modernos.
- Análise de como os SGBDs garantem a integridade e disponibilidade das informações estratégicas para as empresas.
- Familiarizar-se com o SQL Server Management Studio (SSMS) e seus componentes principais.

2. Recursos necessários

- Computadores conectados à internet.
- Material de apoio: livro-texto e videoaulas.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Boas-vindas aos alunos e apresentação do tema e dos objetivos da aula.
- Discussão inicial: "O que é um banco de dados?".
- Exibição de imagens ou vídeo sobre o uso do SQL Server Management Studio (SSMS) e seus componentes principais

3.2. Revisão conceitual (20 minutos)

- Introdução Prática ao Ambiente: Apresentação do SQL Server Management Studio (SSMS).
- Criação de um banco de dados de exemplo (vazio) e familiarização com a interface.

4. Atividade prática (70 minutos)

4.1. Introdução

As atividades de laboratório da disciplina de Banco de dados serão realizadas em primeiro momento com o uso do software brModelo, que será utilizado para a modelagem do banco de dados que será criado. Este software é gratuito e pode ser usado nos computadores dos laboratórios ou, caso queira, o aluno pode utilizá-lo em sua própria máquina. Existe a opção de download no endereço: https://sourceforge.net/projects/brmodelo/files/latest/download, também gratuito, onde os projetos criados poderão ser salvos para utilização futura.

Estudo de caso (e.g., sistema de biblioteca). Utilização de ferramenta CASE (como o SQLServer ou diagramas do SSMS) para criar o Modelo Entidade-Relacionamento (MER).

Crie um estudo de caso utilizando a ferramenta brModelo de modelagem de banco de dados para um estudo de um sistema de gerenciamento de um aplicativo para adoções de pets, crie os modelos conceituais, lógicos e físicos, adicionando os exemplos reais com a utilização das ferramentas brModelo e SQL Server.

O estudo de caso deve apresentar o desenvolvimento completo de um banco de dados para um aplicativo de adoção de animais, desde a concepção inicial até a implementação física, utilizando as ferramentas brModelo para modelagem e SQL Server para implementação, e demonstrando cada etapa do processo de design de banco de dados com exemplos práticos e aplicáveis ao mundo real.

4.2 Levantamento de Requisitos do Sistema

Antes de iniciar a modelagem, é fundamental compreender profundamente as necessidades do sistema. Nosso aplicativo de adoção de pets precisa gerenciar diversos aspectos complexos: cadastro de animais com suas características específicas, histórico médico completo, perfis de adotantes com suas preferências, ONGs parceiras que disponibilizam animais, processos de adoção com suas etapas, e sistema de comunicação entre todas as partes envolvidas.

Gestão de Animais

- Cadastro completo com fotos
- Características físicas e comportamentais
- Histórico médico e vacinas
- Status de disponibilidade

Perfil de Adotantes

- Dados pessoais e contato
- Preferências de adoção
- Histórico de interações
- Documentação necessária

ONGs Parceiras

- Cadastro institucional
- Localização e contato
- Animais sob responsabilidade
- Critérios de adoção

Processo de Adoção

- Solicitações de adoção
- Etapas de aprovação
- Documentação envolvida
- Acompanhamento pós-adoção

4.3. Desenvolvimento do Modelo Conceitual no brModelo

O modelo conceitual representa a visão de mais alto nível do banco de dados, focando nas entidades principais e seus relacionamentos sem se preocupar com detalhes de implementação. Utilizando o brModelo, ferramenta brasileira amplamente adotada no ensino de modelagem de dados, criamos um Diagrama Entidade-Relacionamento (DER), que captura a essência do nosso sistema de adoção.

No brModelo, é necessário identificar as entidades fundamentais: ANIMAL, USUARIO, ONG, VETERINARIO, ADOCAO, e VACINA. Cada entidade é representada por um retângulo, e os relacionamentos por losangos conectando as entidades. Por exemplo, a entidade USUARIO se relaciona com ANIMAL através do relacionamento "solicita_adocao", que posteriormente se materializa na entidade ADOCAO quando aprovada.

1	**Entidades Principais** **ANIMAL:** Representa cada pet disponível para adoção **USUARIO:** Pessoas interessadas em adotar **ONG:** Organizações que disponibilizam animais **VETERINARIO:** Profissionais responsáveis pelo cuidado
2	**Relacionamentos Chave** **possui:** ONG possui ANIMAL (1:N) **realiza_adocao:** USUARIO realiza ADOCAO de ANIMAL (N:N) **atende:** VETERINARIO atende ANIMAL (N:N) **recebe:** ANIMAL recebe VACINA (N:N)
3	**Cardinalidades** Cada ONG pode ter múltiplos animais (1:N) Um usuário pode adotar vários animais ao longo do tempo (1:N) Um animal pode ter histórico de várias consultas veterinárias (N:N) Um animal recebe múltiplas vacinas em sua vida (N:N)

4.4. Transformação para o Modelo Lógico

A transição do modelo conceitual para o lógico envolve a conversão das entidades e relacionamentos em estruturas que podem ser implementadas em um banco de dados relacional. No brModelo, utilizamos a funcionalidade de conversão automática, mas é essencial revisar e ajustar o resultado para garantir a integridade e eficiência do modelo.

Nesta etapa, cada entidade se torna uma tabela, os atributos viram colunas, e os relacionamentos são implementados através de chaves estrangeiras. Relacionamentos N:N requerem tabelas associativas intermediárias. Por exemplo, o relacionamento entre ANIMAL e VACINA gera a tabela HISTORICO_VACINA, que contém as chaves estrangeiras para ambas as tabelas e atributos adicionais como data de aplicação e próxima dose.

Definimos também as chaves primárias (PK) e estrangeiras (FK), tipos de dados para cada atributo e restrições de integridade. A chave primária da tabela ANIMAL é id_animal (INT), enquanto id_ong é uma chave estrangeira referenciando a tabela ONG. Todas as decisões de design nesta fase impactam diretamente a performance e manutenibilidade do sistema final.

Tabela ANIMAL

PK: id_animal, FK: id_ong, Atributos: nome, especie, raca, idade, sexo, porte, cor, descricao, data_cadastro, status

Tabela USUARIO

PK: id_usuario, Atributos: nome, cpf, email, telefone, endereco, cidade, estado, data_nascimento, profissao.

Tabela ONG

PK: id_ong, Atributos: nome_fantasia, cnpj, endereco, telefone, email, responsavel, capacidade_animais

4.5. Desafio inicial - Modelo Físico e Implementação no SQL Server

O modelo físico representa a implementação concreta do banco de dados no SQL Server. Nesta etapa, traduzimos o modelo lógico em comandos DDL (Data Definition Language) que criam efetivamente as estruturas no servidor. Além das tabelas básicas, adicionamos índices para otimização de consultas, constraints para garantir integridade referencial, triggers para automatizar processos, e views para simplificar consultas complexas.

No SQL Server Management Studio, crie o database e configurando suas propriedades. Em seguida, crie as tabelas seguindo uma ordem que respeita as dependências de chaves estrangeiras: primeiro as tabelas independentes (ONG, VETERINARIO), depois as dependentes (ANIMAL), e finalmente as tabelas associativas (ADOCAO, HISTORICO_VACINA, CONSULTA_VETERINARIA).

Exemplo:

```
-- Criação do Database
CREATE DATABASE AdocaoPets
COLLATE Latin1_General_CI_AS;
GO

USE AdocaoPets;
GO

-- Tabela ONG
CREATE TABLE ONG (
    id_ong INT IDENTITY(1,1) PRIMARY KEY,
    nome_fantasia VARCHAR(200) NOT NULL,
    cnpj CHAR(14) UNIQUE NOT NULL,
    endereco VARCHAR(300),
    cidade VARCHAR(100),
    estado CHAR(2),
    telefone VARCHAR(20),
    email VARCHAR(150),
    responsavel VARCHAR(200),
    capacidade_animais INT,
    data_cadastro DATETIME DEFAULT GETDATE()
);
```

Criação das Tabelas

Definição de estruturas, tipos de dados e constraints.

Chaves e Relacionamentos

Implementação de PKs, FKs e índices.

5. Encerramento e orientações finais (20 minutos)

- Discussão sobre a ferramenta brModelo e seu uso, suas vantagens e eventuais dificuldades de uso.
- Dicas para a organização dos componentes e como conectá-los.
- Orientação sobre a entrega do relatório final.

6. Orientações para o Relatório Final

Cada aluno deve produzir um relatório curto contendo:

- Todos os comandos e prints de tela com todos os modelos conceituais, lógicos e físicos.
- Explicação sobre a evolução dos bancos de dados
- **Código-fonte comentado**: Explicação do funcionamento de cada trecho de código implementado na criação do modelo físico.

7. Critérios de avaliação

Critério	Peso	Descrição
Criação dos dicionários de dados, seguindo a regra de negócios aplicativo para adoção de pets	2,0	Compreensão e clareza na teoria de banco de dados
Criação do modelo conceitual, seguindo a regra de negócios aplicativo para adoção de pets	4,0	Compreensão e clareza no uso do brModelo
Criação do modelo lógico, seguindo a regra de negócios aplicativo para adoção de pets	4,0	Capacidade de análise e dedução a partir modelo conceitual criado

Nota final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não cumprirem os requisitos mínimos de funcionamento do código ou não entregarem o relatório dentro do prazo terão sua nota diminuída proporcionalmente.

8. Conclusão

Nesta aula, os alunos entenderam os conceitos fundamentais do uso ferramenta de modelagem na prática do projeto. Além disso, a atividade reforçou que a utilização da ferramenta deve sempre estar alinhada às regras de negócio do sistema.

Bom estudo e boa prática!

Instituto de Ciências Exatas e Tecnológicas	**Disciplina:** Banco de Dados **Título da Aula:** Estudo Prático - Comandos DDL	**ROTEIRO 2**

Roteiro da aula prática – Construindo o modelo físico.

1. Objetivos da aula

- Construir o modelo físico a partir da regra de negócio atribuída no roteiro anterior.
- Criação dos scripts.
- Compreender os fundamentos da criação de um banco de dados com suas respectivas tabelas.

2. Recursos necessários

- Software SQLServer.
- Material de apoio: livro-texto e videoaulas.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Boas-vindas aos alunos e apresentação do tema e dos objetivos da aula.
- Discussão inicial: "Utilizando os comandos DDL".
- Exibição de estudos de caso com a utilização dos principais comandos.

3.2. Revisão conceitual (30 minutos)

- Definição de entidade.
- Atributos.
- Domínios.
- Identificadores (chaves primárias e estrangeiras).
- Atributos monovalorados.
- Atributos multivalorados.

4. Atividade prática (60 minutos)

4.1. Introdução

Parte importante da cardinalidade nos bancos de dados relacionais. Um **relacionamento** é uma associação entre duas ou mais entidades. Por exemplo, podemos ter o relacionamento em que Roberto trabalha no departamento de farmácia. Como com as entidades, podemos desejar reunir um conjunto de relacionamentos semelhantes em um **conjunto de relacionamentos**. Um conjunto de relacionamentos pode ser considerado um conjunto de n-tuplas.

Um outro objetivo desta atividade é demonstrar, por meio do SGBD SQLServer, como podemos criar relacionamento entre as entidades.

4.2. Desafio inicial

Para tornar a prática dinâmica no SQL Server, utilizaremos um cenário de um **Sistema de Gestão de Livraria Digital**.

A cardinalidade define o número de ocorrências de uma entidade que podem estar associadas a uma ocorrência de outra entidade. Matematicamente, tratamos isso como um conjunto de n-tuplas:

Parte I – Relacionamento 1:N (Um para Muitos)

- Identificar e implementar relacionamentos **1:1**, **1:N** e **N:N**.
- Praticar a criação de chaves primárias (PK) e estrangeiras (FK).
- Utilizar o SQL Server para garantir a integridade referencial.

Esboçar o circuito para cada uma das expressões, simulá-lo e completar a tabela verdade.

Cenário: Cada **Editora** pode publicar vários livros, mas um **Livro** específico é publicado por apenas uma editora.

- **Tarefa:** Criar as tabelas Editoras e Livros.
- **Ponto de Atenção:** A FK (Foreign Key) deve ficar no lado "N" da relação (em Livros).

Parte II – Relacionamento N:N (Muitos para Muitos)

Cenário: Um **Livro** pode ter vários **Autores**, e um **Autor** pode escrever vários livros.

- **Tarefa:** Criar a tabela Autores e uma **tabela associativa (tabela de ligação)** chamada Livro_Autor.
- **Ponto de Atenção:** Relacionamentos N:N não existem fisicamente em bancos relacionais; eles são decompostos em dois relacionamentos 1:N.

Parte III – Relacionamento 1:1 (Um para Um)

Cenário: Por questões de segurança e performance, os **Detalhes Extras** de um livro (como o resumo em PDF ou metadados pesados) são armazenados em uma tabela separada. Cada livro tem exatamente um registro de detalhes.

- **Tarefa:** Criar a tabela Detalhes_Livro.
- **Ponto de Atenção:** Para garantir que seja 1:1, a chave estrangeira em Detalhes_Livro também deve ser a chave primária ou possuir uma restrição de UNIQUE.

4.3. Ampliação do desafio

Imagine que o sistema agora precisa permitir que **clientes** comprem esses livros.

1. Um cliente pode fazer vários pedidos.
2. Um pedido pode ter vários livros.

5. Encerramento e orientações finais (20 minutos)

- Discussão sobre como deve ser aplicado o conceito de cardinalidade.
- Dicas de como definir e implementar relacionamentos **1:1**, **1:N** e **N:N**.
- Orientação sobre a entrega do relatório final.

6. Orientações para o Relatório Final

Cada aluno deve produzir um relatório curto contendo:

- Scripts com todos os assuntos elencados.

7. Critérios de avaliação

Critério	Peso	Descrição
Apresentação relacionamentos entre as entidades com suas respectivas cardinalidades: **1:1**, **1:N** e **N:N**	3,5	Compreensão e clareza na teoria do uso de cardinalidade
Apresentação dos scripts	3,5	Compreensão e clareza no uso da cardinalidade usando o sql server
Desafio final	3,0	Printar e incluir no documento todos os resultados da cardinalidade obtido a partir do scripts do SQL Server.

Nota final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não cumprirem os requisitos mínimos de funcionamento do código ou não entregarem o relatório dentro do prazo terão sua nota diminuída proporcionalmente.

8. Conclusão

Nesta aula, os alunos entenderam os conceitos fundamentais da construção de modelo lógicos e os fundamentos do uso da cardinalidade. Além disso, a atividade consolidou na prática as habilidades da programação SQL no software SQL Server.

Bom estudo e boa prática!

Instituto de Ciências Exatas e Tecnológicas	**Disciplina:** Banco de Dados **Título da Aula:** Estudo Prático - Comandos DML - Parte I	**ROTEIRO 3**

Roteiro da aula prática – Criando programação SQL com DML.

1. Objetivos da aula

- Analisar os principais comandos DML e sua utilização no dia a dia.

2. Recursos necessários

- Computadores conectados à internet.
- Material de apoio: livro-texto e videoaulas.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Boas-vindas aos alunos e apresentação do tema e dos objetivos da aula.
- Discussão inicial: "Como podemos manipular dados em um banco de dados e como eles podem ser inseridos, alterados, pesquisados e excluídos quando necessário".
- Exibição prática da execução dos comandos.

3.2. Revisão conceitual (30 minutos)

- Banco de dados modelo lógico.
- Entidades, atributos, tipos de dados.
- Dependência de relacionamento.

4. Atividade prática (60 minutos)

4.1. Introdução

Nessa aula vamos introduzir os conceitos de DML (Data Manipulation Language), que são os comandos responsáveis pela interação com os dados dentro das tabelas.

- INSERT: Como adicionar novas linhas (pedidos, clientes, pizzas).
- UPDATE: Como modificar informações existentes (alterar status de entrega ou preços).
- DELETE: Como remover registros (cancelar um item ou limpar dados obsoletos).
- SELECT (DML/DQL): Embora técnico, usaremos o básico para verificar se nossas alterações funcionaram.

4.2. Desafio inicial

Imagine que a "Bella Napoli" acabou de inaugurar. O banco de dados já possui as tabelas Clientes, Pizzas e Pedidos, mas elas estão vazias ou desatualizadas. Seu objetivo é fazer a gestão desses dados.

- Insira 3 tipos de pizzas (Ex.: Margheritta, Calabresa, Portuguesa) na tabela Pizzas.
- Cadastre 2 clientes novos na tabela Clientes.
- Crie um novo registro na tabela Pedidos vinculando um cliente a uma pizza.

4.3. Ampliação do desafio

O negócio cresceu e surgiram imprevistos. Use comandos avançados para resolver:

1. **Reajuste de Preços:** Devido à alta do queijo, aumente o preço de todas as pizzas em 10% usando uma expressão matemática no UPDATE.
2. **Fidelidade:** O cliente que mais comprou mudou de endereço. Atualize o logradouro dele.
3. **Cancelamento de Segurança:** Remova todos os pedidos que foram marcados com o status "Cancelado" para limpar o banco.
4. **Promoção Relâmpago:** Atualize o nome da "Pizza da Casa" para "Super Promoção" e reduza o preço apenas desta unidade específica.

5. Encerramento e orientações finais (20 minutos)

- **Consistência de Dados:** Discutiremos porque o SQL impede você de deletar uma pizza que está vinculada a um pedido ativo (Integridade Referencial).
- **Boas Práticas:** O uso do SELECT antes do DELETE para conferir o que será apagado.
- **Dica de Ouro:** Sempre use a cláusula WHERE em comandos de alteração.

6. Orientações para o Relatório Final

Cada aluno deve produzir um relatório curto contendo:

1. **Scripts SQL:** Todos os comandos utilizados durante a prática.
2. **Screenshots:** Evidência da execução de cada comando e o resultado final nas tabelas (o "antes e depois").
3. **Análise Crítica:** Um parágrafo explicando qual comando foi mais desafiador e por quê.

7. Critérios de avaliação

Critério	Peso	Descrição
Sintaxe SQL	3,0	Evidências do código executável sem erros e no padrão ANSI
Lógica de Filtro	3,0	Uso correto da cláusula WHERE para manipulação precisa
Organização e Resolução de Problemas	2,0	Relatório claro, com capturas de tela legíveis com comentários. Capacidade de aplicar o reajuste percentual de 10% corretamente.
Desafio final	2,0	Entrega do relatório final

Nota final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não cumprirem os requisitos mínimos de funcionamento do código ou não entregarem o relatório dentro do prazo terão sua nota diminuída proporcionalmente.

8. Conclusão

Nesta aula, os alunos entenderam os conceitos fundamentais do uso conceitos de **DML (Data Manipulation Language)**, que são os comandos responsáveis pela interação com os dados dentro das tabelas, bem como reforçaram a utilização prática da linguagem SQL.

Bom estudo e boa prática!

Instituto de Ciências Exatas e Tecnológicas	**Disciplina:** Banco de Dados **Título da Aula:** Estudo Prático - Comandos DML - Parte II	**ROTEIRO 4**

Roteiro da aula prática – Criando programação SQL com DML – Parte II.

1. Objetivos da aula

Nesta etapa, revisaremos os fundamentos do modelo lógico e os comandos de manipulação. Abordaremos os seguintes tópicos:

- Entidades e Atributos: Identificação das tabelas (Pizzas, Clientes, Pedidos) e seus campos.
- Comandos DML fundamentais:
  - INSERT: Inserção de novos registros.
  - UPDATE: Modificação de dados existentes.
  - DELETE: Remoção de registros.
  - SELECT: Consulta e verificação de alterações.
- Operadores para Consultas e Filtros:
  - Aritméticos: +, -, *, / (usados para cálculos de preços e descontos).
  - Relacionais: =, <>, >, <, >=, <= (usados para comparar valores).
  - Lógicos: AND, OR, NOT (usados para combinar múltiplas condições no WHERE).
- Dependência de Relacionamento: Entender como as chaves estrangeiras conectam pedidos a clientes e pizzas.

2. Recursos necessários

- Computadores conectados à internet.
- Material de apoio: livro-texto e videoaulas.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Boas-vindas aos alunos e apresentação do tema e dos objetivos da aula.
- Discussão inicial: "Como podemos manipular dados em um banco de dados e como eles podem ser inseridos, alterados, pesquisados e excluídos quando necessário".
- Exibição prática da execução dos comandos.

3.2. Revisão conceitual (30 minutos)

- Banco de dados modelo lógico principais comandos DML.
- Uso eficaz do WHERE.
- Consultas de verificação utilizando a lógica de comparação e combinação de condições.

4. Atividade prática (60 minutos)

4.1. Introdução

A linguagem DML permite interagir diretamente com a massa de dados da pizzaria. Utilizaremos o SELECT para validar cada operação realizada.

4.2. Desafio inicial

Você deve popular o banco de dados e realizar consultas de verificação utilizando a lógica de comparação e combinação de condições.

Tarefa 1: Inserção com Verificação Relacional

1. **Popular o Cardápio:** Insira 3 pizzas (Margheritta, Calabresa e Portuguesa) com preços variados (ex.: R$ 45,00, R$ 55,00 e R$ 65,00).
2. **Consulta com Operador Relacional:** Após inserir, utilize o comando SELECT com o operador **maior que** ($>$) para listar apenas as pizzas que custam mais de R$ 50,00.

4.3. Ampliação do desafio

1. **Cadastrar Clientes:** Insira 2 clientes, um residente no "Bairro Centro" e outro no "Bairro Sul".
2. **Consulta com Operador Lógico (AND/OR):** * Tente localizar um cliente que more no "Bairro Centro" **E** cujo ID seja igual a 1.
    - Liste todos os clientes que moram no "Bairro Sul" **OU** no "Bairro Norte".

Tarefa 3: Registro de Venda Condicional

1. **Vincular Pedido:** Crie um registro na tabela Pedidos associando um cliente a uma pizza.
2. **Verificação de Status:** Utilize o operador relacional **diferente de** (<> ou !=) para selecionar todos os pedidos que **não** estão com o status "Finalizado".

5. Encerramento e orientações finais (20 minutos)

- **Integridade Referencial:** Discussão sobre porque o sistema impede a exclusão de uma pizza que possui pedidos ativos.
- **Segurança na Operação:** A importância de sempre utilizar a cláusula WHERE para evitar alterações indesejadas em toda a base de dados.
- **Validação:** O hábito de realizar um SELECT antes de um DELETE para garantir que o filtro está correto.

6. Orientações para o Relatório Final

Cada aluno deve produzir um relatório curto contendo:

- **Scripts SQL:** Todos os códigos de INSERT, UPDATE, DELETE e SELECT desenvolvidos.
- **Screenshots:** Capturas de tela mostrando o estado das tabelas "antes" e "depois" de cada operação relevante.
- **Análise Crítica:** Um breve parágrafo sobre qual operador ou comando foi mais complexo de aplicar e o porquê.

7. Critérios de avaliação

Critério	Peso	Descrição
Sintaxe SQL	3,0	Código executável sem erros no padrão ANSI
Lógica de Filtro	3,0	Uso correto da cláusula WHERE e operadores lógicos/relacionais
Organização	2,0	Relatório claro com capturas legíveis e comentários
Desafio final	2,0	Entrega completa do relatório dentro do prazo e requisitos

Nota final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não cumprirem os requisitos mínimos de funcionamento do código ou não entregarem o relatório dentro do prazo terão sua nota diminuída proporcionalmente.

8. Conclusão

Nesta aula, os alunos reforçaram o uso de consultas de verificação utilizando a lógica de comparação e combinação de condições.

Bom estudo e boa prática!

Instituto de Ciências Exatas e Tecnológicas	**Disciplina:** Banco de Dados **Título da Aula:** Operadores especiais: BETWEEN, IS NULL, LIKE, IN e EXISTS	**ROTEIRO 5**

Roteiro da aula prática – Utilização dos operadores especiais: BETWEEN, IS NULL, LIKE, IN e EXISTS.

1. Objetivos da aula

- Entender o funcionamento dos **Operadores especiais: BETWEEN, IS NULL, LIKE, IN e EXISTS**.
- A linguagem SQL do padrão ANSI permite a utilização de operadores especiais com a cláusula WHERE. Esses operadores incluem:
- **BETWEEN**: utilizado para verificar se o valor de um atributo está dentro de uma faixa.
- **IS NULL**: utilizado para verificar se o valor de um atributo é nulo.
- **LIKE**: utilizado para verificar se o valor de um atributo coincide com um determinado padrão de caractere.
- **IN**: utilizado para verificar se o valor de um atributo coincide com qualquer valor de uma lista.
- **EXISTS**: utilizado para verificar se uma subconsulta retorna alguma linha.

2. Recursos necessários

- Computadores conectados à internet.
- Material de apoio: livro-texto e videoaulas.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Boas-vindas aos alunos e apresentação do tema e dos objetivos da aula.
- Discussão inicial: "O que são operadores especiais e para que são indicados?".

3.2. Revisão conceitual (30 minutos)

Nesta etapa, faremos uma revisão do modelo lógico e introduziremos os operadores especiais que tornam o comando SELECT e as cláusulas de filtragem (WHERE) mais poderosas:

- **BETWEEN:** Define um intervalo de valores (ex.: preços entre R$ 30 e R$ 50).
- **IS NULL / IS NOT NULL:** Verifica a ausência ou presença de informação em um campo (ex.: pedidos sem entregador atribuído).
- **LIKE:** Realiza buscas por padrões de texto usando caracteres curinga como % (ex.: nomes de pizzas que começam com "Calab").
- **IN:** Verifica se um valor pertence a uma lista específica (ex.: pizzas dos sabores "Muçarela", "Calabresa", "Margheritta").
- **EXISTS:** Testa a existência de registros em uma subconsulta (ex.: clientes que já realizaram pelo menos um pedido).

4. Atividade prática (60 minutos)

4.1. Introdução

Nesta aula, introduziremos os comandos DML (**INSERT**, **UPDATE**, **DELETE**) e utilizaremos o **SELECT** para validar a interação com os dados da pizzaria "Bella Napoli".

4.2. Desafio inicial

O banco de dados possui as tabelas Clientes, Pizzas e Pedidos. Realize as seguintes tarefas:

1. **Inserção de Dados:** Insira 5 pizzas com preços variando de R$ 35,00 a R$ 85,00.
2. **Busca por Padrão (LIKE):** Liste todas as pizzas cujo nome contenha a palavra "Frango".
3. **Filtro de Faixa (BETWEEN):** Selecione as pizzas que custam entre R$ 40,00 e R$ 60,00.
4. **Seleção por Lista (IN):** Recupere os dados dos clientes que moram nos bairros "Centro", "Sudoeste" ou '"Jardins".

.

4.3. Ampliação do desafio

O negócio cresceu e agora precisamos de consultas mais complexas para a gestão:

1. **Gestão de Dados Ausentes (IS NULL):** Identifique todos os pedidos onde o campo data_entrega ainda está nulo (pedidos pendentes).
2. **Validação de Existência (EXISTS):** Liste os nomes dos clientes que possuem registros na tabela de Pedidos.
3. **Atualização em Lote (IN):** Aplique um reajuste de 10% apenas nas pizzas que pertencem às categorias ("Doces", "Especiais").
4. **Limpeza Seletiva (LIKE):** Remova registros de testes onde o nome do cliente comece com "TESTE%".

5. Encerramento e orientações finais (20 minutos)

- **Consistência e Integridade:** Discussão sobre a impossibilidade de deletar registros vinculados a pedidos ativos (Integridade Referencial).
- **Boas Práticas:** Reforço sobre o uso do SELECT antes de qualquer DELETE para validar o filtro.
- **Dica de Ouro:** A importância vital da cláusula WHERE para evitar a manipulação acidental de toda a tabela.

6. Orientações para o Relatório Final

Cada aluno deve produzir um relatório curto contendo:

- **Scripts SQL:** Comandos utilizados, incluindo os operadores especiais solicitados.
- **Screenshots:** Evidências do "antes e depois" da execução.
- **Análise Crítica:** Um parágrafo sobre qual operador foi mais útil para a gestão da pizzaria.

7. Critérios de avaliação

Critérios de avaliação

Critério	Peso	Descrição
Sintaxe SQL	3,0	Código executável sem erros no padrão ANSI
Lógica de Filtro	3,0	Uso correto de WHERE e operadores especiais (LIKE, IN, etc.)
Organização	2,0	Relatório claro com capturas legíveis e comentários
Desafio final	2,0	Entrega completa do relatório dentro do prazo e requisitos

Nota final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não cumprirem os requisitos mínimos de funcionamento do código ou não entregarem o relatório dentro do prazo terão sua nota diminuída proporcionalmente.

8. Conclusão

Nesta aula, os alunos reforçaram o uso prático de consultas de verificação utilizando a lógica de comparação e combinação de condições e fazendo o uso estratégico de operadores especiais.

Bom estudo e boa prática!

Instituto de Ciências Exatas e Tecnológicas	**Disciplina:** Banco de Dados **Título da Aula:** Criação de views	**ROTEIRO 6**

Roteiro da aula prática – Criação de views

Este roteiro de estudo prático foca na criação e gestão de **Views** (Visões) em SQL. As views funcionam como "tabelas virtuais" que permitem simplificar consultas complexas, aumentar a segurança e organizar a lógica de negócios da nossa pizzaria "Bella Napoli".

1. Objetivos da aula

- Explicar por que as views permitem "esconder" a complexidade de consultas que envolvem múltiplos relacionamentos e cálculos.
- Explicar por que as views podem ser um filtro de segurança.
- Como funciona a independência e consistência de Dados.

2. Recursos necessários

- Computadores conectados à internet.
- Material de apoio: livro-texto e videoaulas.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Boas-vindas aos alunos e apresentação do tema e dos objetivos da aula.
- Discussão inicial: "O que é uma view e como ela pode ser utilizada no dia a dia?".

3.2. Revisão conceitual (30 minutos)

O que é uma View: Uma consulta SELECT armazenada no banco de dados que pode ser consultada como se fosse uma tabela.

Sintaxe Básica: Uso do comando CREATE VIEW associado ao AS SELECT.

Vantagens: * Simplificação: Esconder a complexidade de múltiplos JOINs.

- **Segurança:** Mostrar apenas as colunas necessárias para certos usuários (ex.: esconder dados sensíveis de clientes).
- Consistência: Garantir que todos usem a mesma regra de negócio (ex.: cálculo de lucro).

Views Atualizáveis vs. Somente Leitura: Limitações ao tentar usar comandos DML (INSERT, UPDATE) através de uma view.

4. Atividade prática (60 minutos)

4.1. Introdução

Na "Bella Napoli", os gestores precisam de relatórios rápidos sem ter que escrever códigos complexos toda vez. As views serão criadas sobre as tabelas base de Clientes, Pizzas e Pedidos para facilitar essa visualização.

4.2. Desafio inicial

O atendente precisa de uma lista rápida de pizzas prontas para venda.

1. **Criação da View de Preços:** Crie uma view chamada vw_cardapio_economico que selecione apenas o nome e o preço das pizzas que custam menos de R$ 50,00.
2. **Consulta Simples:** Utilize um SELECT * na view recém-criada para verificar se o filtro foi aplicado corretamente.
3. **Mascaramento de Dados:** Crie uma view chamada vw_clientes_contato que exiba apenas o nome do cliente e o telefone, ocultando o endereço por questões de privacidade.

4.3. Ampliação do desafio

O gerente precisa de visões que combinem dados de diferentes tabelas.

1. **Relatório de Pedidos Ativos:** Crie uma view chamada vw_pedidos_pendentes que junte (JOIN) os dados do cliente e o nome da pizza para todos os pedidos que não tenham o status "Finalizado" ou "Cancelado".
2. **Visão de Faturamento:** Crie uma view que utilize operadores aritméticos para mostrar o nome da pizza, o preço original e uma coluna calculada com o "Preço com Taxa de Entrega" (Preço + 10%).
3. **Manutenção:** Utilize o comando OR REPLACE para modificar uma das views anteriores, adicionando uma nova coluna de data.

5. Encerramento e orientações finais (20 minutos)

- **Persistência:** Discussão sobre o fato de que a view não armazena dados próprios, mas reflete os dados das tabelas originais em tempo real.
- **Desempenho:** Breve explicação sobre como views complexas podem impactar a performance se não forem bem planejadas.
- **Boas Práticas:** Nomear as views com prefixos (como vw_) para diferenciá-las de tabelas físicas no dicionário de dados.

6. Orientações para o Relatório Final

Cada aluno deve produzir um relatório curto contendo:

- Scripts SQL: **Os comandos de criação de todas as views solicitadas.**
- Screenshots: **Evidência do resultado da consulta SELECT realizada em cada view.**
- Análise Crítica: **Um parágrafo explicando por que o uso de views é mais seguro do que dar acesso direto às tabelas para um funcionário administrativo.**

7. Critérios de avaliação

Critério	Peso	Descrição
Sintaxe SQL	3,0	Uso correto de CREATE VIEW e subconsultas sem erros de execução
Lógica de Filtro	3,0	Aplicação correta de filtros (WHERE) e cálculos aritméticos dentro das views
Organização	2,0	Relatório bem estruturado com imagens nítidas das consultas.
Desafio final	2,0	Capacidade de realizar o JOIN entre tabelas dentro de uma visão

Nota final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não cumprirem os requisitos mínimos de funcionamento do código ou não entregarem o relatório dentro do prazo terão sua nota diminuída proporcionalmente.

8. Conclusão

Nesta aula, os alunos reforçaram o uso prático de consultas com views utilizando a lógica de comparação e combinação de condições e fazendo o uso estratégico das views para agilizar mecanismos de consultas sem prejudicar o funcionamento do banco de dados.

Bom estudo e boa prática!

Instituto de Ciências Exatas e Tecnológicas	**Disciplina:** Banco de Dados **Título da Aula:** Stored Procedures (Procedimentos Armazenados)	**ROTEIRO 7**

Roteiro da aula prática – Criando procedures.

Este roteiro de estudo prático foca na criação e execução de **Stored Procedures** (Procedimentos Armazenados), que permitem automatizar processos repetitivos e encapsular lógicas complexas de negócio dentro do banco de dados da nossa pizzaria "Bella Napoli".

1. Objetivos da aula

- Compreender o conceito de Stored Procedures e sua importância na automação de processos.
- Aprender a sintaxe de criação, execução e passagem de parâmetros.
- Desenvolver lógicas procedimentais para otimizar o atendimento da pizzaria.

2. Recursos necessários

- Computadores conectados à internet.
- Material de apoio: livro-texto e videoaulas.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Boas-vindas aos alunos e apresentação do tema e dos objetivos da aula.
- Discussão inicial: "Como criar procedures de forma efetiva".

3.2. Revisão conceitual (30 minutos)

O que é uma Procedure: Um bloco de código SQL, salvo que pode ser executado repetidamente.

Parâmetros de Entrada (IN): Como passar dados para dentro da procedure (ex.: ID do pedido).

Parâmetros de Saída (OUT): Como retornar valores calculados (ex.: total da conta).

Vantagens: Redução do tráfego de rede, melhor performance e centralização da regra de negócio.

Diferença entre Procedure e View: Procedures podem realizar ações de alteração de dados (DML), enquanto Views são apenas consultas.

4. Atividade prática (60 minutos)

4.1. Introdução

Na "Bella Napoli", certas tarefas ocorrem centenas de vezes por dia, como registrar um pagamento ou atualizar o estoque. Em vez de escrever vários comandos manuais, criaremos procedures para automatizar essas ações.

4.2. Desafio inicial – Automatizando o Pedido

Para agilizar o atendimento, você deve facilitar a inserção de novos pedidos.

1. **Criação da Procedure:** Desenvolva uma procedure chamada sp_registrar_pedido que receba o id_cliente e o id_pizza como parâmetros de entrada.
2. **Lógica Interna:** A procedure deve inserir automaticamente uma nova linha na tabela Pedidos com o status "Em Preparação" e a data atual.
3. **Execução:** Utilize o comando CALL para registrar 3 pedidos diferentes e valide o resultado com um SELECT.

4.3. Ampliação do desafio – Gestão financeira

O gerente quer automatizar o encerramento da conta e aplicação de descontos.

1. **Procedure de Fechamento:** Crie a procedure sp_fechar_conta que recebe o id_pedido.
2. **Cálculo Automático:** A procedure deve buscar o preço da pizza associada e atualizar o status do pedido para "Finalizado".
3. **Lógica de Fidelidade:** Adicione uma condição: se o cliente for "VIP" (use um parâmetro ou consulta), a procedure deve aplicar automaticamente um desconto de 10% no valor final utilizando operadores aritméticos.

5. Encerramento e orientações finais (20 minutos)

Manutenibilidade: Discussão sobre como é mais fácil alterar uma regra de negócio dentro da procedure do que em todo o sistema da pizzaria.

Segurança: Reforço de que usuários podem ter permissão para executar uma procedure sem ter acesso direto para deletar dados das tabelas base.

Boas Práticas: O uso de comentários dentro do código da procedure para documentar sua função.

6. Orientações para o Relatório Final

Cada aluno deve produzir um relatório curto contendo:

Scripts SQL: O código completo das procedures criadas.

Screenshots: Evidência do comando CALL sendo executado e o impacto nas tabelas de Pedidos e Pizzas.

Análise Crítica: Um parágrafo explicando como a procedure facilitaria o trabalho do pizzaiolo no dia a dia.

7. Critérios de avaliação

Critério	Peso	Descrição
Sintaxe SQL	3,0	Uso correto de CREATE PROCEDURE, BEGIN e END
Lógica de parâmetros	3,0	Definição e uso correto de parâmetros IN e OUT
Organização	2,0	Relatório claro com capturas de tela e comentários explicativos.
Desafio final	2,0	Implementação correta da lógica de desconto no fechamento da conta

Nota final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não cumprirem os requisitos mínimos de funcionamento do código ou não entregarem o relatório dentro do prazo terão sua nota diminuída proporcionalmente.

8. Conclusão

Nesta aula, os alunos entenderam que as Procedures são ferramentas fundamentais para a eficiência e integridade de um banco de dados. Elas permitem que a "Bella Napoli" opere com menos erros manuais, pois as regras de negócio (como o cálculo de descontos ou a mudança de status de um pedido) ficam centralizadas e protegidas dentro do servidor SQL. Além de otimizar a performance, o uso de procedimentos armazenados eleva o nível de segurança, permitindo uma gestão de dados profissional e escalável.

Bom estudo e boa prática!

Instituto de Ciências Exatas e Tecnológicas	**Disciplina:** Banco de Dados **Título da Aula:** Utilizando comando DCL	**ROTEIRO 8**

Roteiro da aula prática – Comandos DCL.

Este roteiro de estudo prático foca em DCL (Data Control Language), os comandos responsáveis pela segurança e controle de acesso aos dados. Diferente do DML ou DQL, aqui o objetivo é definir quem pode fazer o quê no banco de dados da pizzaria "Bella Napoli".

1. Objetivos da aula

- Compreender como são geradas as regras de acesso ao banco de dados.
- Criar usuários e associá-los às devidas permissões.
- Simular o acesso dos usuários com suas restrições de acesso.

2. Recursos necessários

- Computadores conectados à internet.
- Material de apoio: livro-texto e videoaulas.

3. Estrutura da aula

3.1. Abertura (10 minutos)

- Boas-vindas aos alunos e apresentação do tema e dos objetivos da aula.
- Discussão inicial: "Como utilizar os comandos DCL de forma efetiva".

3.2. Revisão conceitual (30 minutos)

Conceito de DCL: Comandos que controlam o acesso aos dados e a autorização dos usuários.

Comando GRANT: Como conceder privilégios específicos (Leitura, Escrita, Exclusão) a funcionários.

Comando REVOKE: Como remover permissões previamente concedidas.

Usuários e Perfis (Roles): A diferença entre criar acessos para um "Gerente" versus um "Atendente".

Segurança e Integridade: Por que restringir o acesso é vital para evitar exclusões acidentais ou vazamento de dados de cliente?

4. Atividade prática (60 minutos)

4.1. Introdução

Na "Bella Napoli", não queremos que o entregador consiga alterar o preço das pizzas ou que o atendente apague o histórico de vendas. Vamos configurar as camadas de acesso para as tabelas Clientes, Pizzas e Pedidos.

4.2. Desafio inicial – Configurando Acessos Básicos

Imagine que você contratou um novo **Atendente** e um **Gerente**.

1. **Criação de Usuários:** Crie dois usuários no banco de dados: atendente_pizzaria e gerente_vendas.
2. **Privilégios de Leitura:** Permita que o atendente_pizzaria possa apenas visualizar (SELECT) as tabelas de Pizzas e Clientes.
3. **Privilégios de Operação:** Permita que o atendente_pizzaria possa inserir novos pedidos (INSERT na tabela Pedidos), mas sem permissão para deletar nada.

4.3. Ampliação do desafio – Gestão Avançada de Segurança

O negócio cresceu e a política de segurança ficou mais rígida:

1. **Controle Total:** Conceda ao gerente_vendas todos os privilégios (ALL PRIVILEGES) sobre todas as tabelas, incluindo a capacidade de alterar preços (DML).
2. **Revogação de Acesso:** O atendente_pizzaria foi promovido. Remova (REVOKE) as permissões limitadas dele para prepará-lo para um novo perfil.
3. **Restrição de Colunas:** (Opcional/Avançado) Tente conceder permissão de UPDATE para o atendente apenas na coluna status da tabela Pedidos, impedindo que ele altere o valor total da venda.

5. Encerramento e orientações finais (20 minutos)

- **Princípio do Privilégio Mínimo:** Discussão sobre dar apenas o acesso necessário para cada função.
- **Auditoria:** Como verificar quais permissões cada usuário possui atualmente.
- **Relação DML/DCL:** Reforçar que, sem o comando GRANT (DCL) correto, nenhum comando de inserção ou alteração (DML) funcionará para o usuário.

6. Orientações para o Relatório Final

Cada aluno deve produzir um relatório curto contendo:

- **Scripts SQL:** Todos os comandos de criação de usuários, GRANT e REVOKE utilizados.
- **Screenshots:** Evidência de tentativas de acesso (incluindo uma captura de erro ao tentar executar um comando sem permissão).
- **Análise Crítica:** Um parágrafo explicando a importância de restringir o comando DELETE na rotina de uma empresa.

7. Critérios de avaliação

Critério	Peso	Descrição
Sintaxe SQL	3,0	Uso correto dos comandos GRANT/REVOKE sem erros
Lógica de permissões	3,0	Atribuição correta de níveis de acesso para cada perfil (Atendente vs Gerente)
Organização	2,0	Relatório claro com evidências de sucesso e erro de permissão.
Desafio final	2,0	Capacidade de verificar acessos a dados de forma segura

Nota final: Será a soma dos valores obtidos em cada critério. Alunos ou equipes que não cumprirem os requisitos mínimos de funcionamento do código ou não entregarem o relatório dentro do prazo terão sua nota diminuída proporcionalmente.

8. Conclusão

Nesta aula, os alunos entenderam os conceitos fundamentais do uso de **DCL**, que são os comandos responsáveis pela segurança e autorização de acesso aos dados, bem como reforçaram a utilização prática da linguagem SQL para o controle de permissões.

Bom estudo e boa prática!